Estimation of epitope residues in protein sequences

Saar Hamam , Michal Elad , Elishay Avram , Ella Levin

Raymond and Beverly Sackler Faculty of Exact Sciences, School of Computer Science, Tel Aviv University

Abstract. In this paper, we propose a deep learning approach for the prediction of epitope regions in protein sequences. Prediction of epitope regions is of great importance to the design and development of vaccines, aimed at inducing antigen-specific antibody production. However, the prediction accuracy of existing models remains marginal and requires further improvement. The method described in the paper is based on artificial recurrent neural networks (RNN). Two architectures of RNN were examined: long short-term memory (LSTM), and gated recurrent units (GRU). LSTMs and GRUs can generally achieve high prediction accuracy; however, epitopes prediction is yet a complex problem to solve. Thus, we will cover several configurations of the network and their impact on the learning process of the network.

1 Introduction

An epitope is a part of an antigen, recognized by the immune system. The epitope is the area to which the antibody binds. Binding between the two occurs only if their structures are complementary. Some antigens may have several distinct epitopes on their surface, each matching a different antibody. Note that this is a highly important factor in our attempts to predict the epitope region, as will be explained later.

There are two types of epitopes of protein antigens, conformational epitopes, and linear epitopes. Here, we will focus on linear epitopes alone. A linear epitope is comprised of contiguous amino acid residues. However, the residues' ability to adopt the epitope's 3-D structure is not determined solely by their primary structure. The surrounding residues, as well as more distant ones, all affect their potential of being an epitope [1].

A substance that mimics the structure and function of an epitope can be considered as a "vaccine" to an organism that is aimed to induce specific antibodies in vivo. Epitope-based vaccine design is the cornerstone of vaccine development. They are considered cost and time-effective and provide maximal therapeutic efficacy with minimal side effects [2]. Therefore, various epitope-focused studies have been conducted to design safe and effective vaccines. Computational immuno-informatics is the basis of epitope vaccine development, and thus the major interest in epitope prediction algorithms.

In this paper, we propose a method for epitope prediction using a long short-term memory network (LSTM). As mentioned previously, the more distant residues affect the epitope prediction as well. Hence, it is crucial to incorporate not only the target amino acid sequence but the long-range features of the protein. This is the reasoning behind the choice of LSTM as the neural network's architecture.

2 Algorithm Description

The input of the neural network is a sequence of amino acids. The output is a probabilities vector of the same length. Each entry in the output vector is a floating number in the range $[0, 1]$ representing the residue's probability of being in an epitope (a value of 1 means epitope and 0 means non-epitope).

2.1 Data Preprocessing

The dataset used for training and testing the network was taken from the ”IEDB Linear Epitope Dataset” database [3]. It is a FASTA file containing observations of epitopes in B-cell antibodies. In it, are over 30,000 protein sequences, i.e., a series of residues. Within each sequence is a marked region, representing an experimentally verified epitope. The dataset was partitioned into train and test datasets with a ratio of 0.7: 0.3, respectively.

As mentioned earlier, an antigen may have several distinct epitopes on its surface. As a result, a single sequence can have multiple appearances in the original dataset (even over 100), in each a different region marked as the epitope. These regions may overlap or may be in utterly different locations in the sequence. Consequently, several issues arose. First, it means that the network might receive a different tag each time, for the same input (it learns the epitope is in a certain region and then another, alternately). Second, some sequences appear numerous times whereas others only once, meaning the training dataset is biased and the network “learns” some sequences more than others. Finally, it implies that an almost identical input can be found in the training dataset and the test dataset. Since there are numerous cases of repeating sequences, it implies that a considerable portion of the test set is very similar to the train set and is therefore not suitable for testing the network. To address these issues, and since this method proved beneficial beforehand [4], we decided to union the repeating sequences to a single representative. The representative marks all regions which were observed as an epitope, in any of the sequence’s appearances. Ultimately, only 3159 unique sequences remained in our dataset after the union procedure.



Fig. 1. An illustration of the sequences’ union procedure, based on the figure shown in [4]. Rectangles of the same color represent appearances of the same sequence when the black hollow rectangle marks the epitope. To the left are the original sequences and to the right are the remaining sequences and epitope verifications.

To enhance the learning process, five numerical features are added to each amino acid. Hence, the input is not only the residue type, i.e. ‘a’ or ‘v’, but also the following: volume [5], hydrophobicity [6], polarity [7], relative surface accessibility (RSA)[8], and secondary structure (SS).

The data for these properties was retrieved from several articles, found in the references section, excluding the secondary structure. The latter is estimated using the Biopyhton module [9], based on a window of 7 residues in the sequence, surrounding the desired residue. The secondary structure obtained for each residue in the sequence is one of three possibilities – “Helix”, “Turn”, or “Sheet”. The estimation relies on the frequency of each amino acid type’s appearance in any of the structures. Note that the secondary structure estimation incorporates the locality effect on the epitope prediction.

Since convergence is usually faster if the mean of each of the input variables from the training set is in the vicinity of zero, we chose to implement numerical features standardization [10]. This was done using the z-score calculation which scales each physical property (e.g., polarity) to have a mean of zero and a variance of a single standard deviation.

As for the secondary structure, each was assigned a numerical value (e.g., “Helix”: 0, “Turn”: 1, “Sheet”: 2) which was then passed on to the network along with the other four numerical features. Given that the secondary structures’ representatives (0,1,2) are arbitrary, and don’t represent any similarity between the structures, this may not be the optimal choice. Presumably, an additional embedding layer for the secondary structure would be beneficial, but due to lack of time we did not manage to do that.

2.2 Network Architectures

In this work we have implemented two different RNN networks, LSTM (long short-term memory network) and GRU (gated recurrent unit). The implementation makes use of the well-known Python module, Pytorch [11], an open-source machine learning framework.

LSTMs and GRUs are known for retaining long term sequences and don't suffer from the vanishing gradients phenomenon typical to other vanilla RNNs. Another reasoning for these choices is the prediction’s dependency on the residue’s neighbors. As mentioned in the introduction, the surrounding residues affect the probability of being in an epitope.

Therefore, we chose to use neural networks (NN) which base their output on the input history as well.

The dataset is comprised of protein sequences in varying lengths. The training process, however, occurs in batches of sequences. This enforces a constraint on the sequences in the batch as they all must be of the same length. To deal with that matter, we padded the shorter inputs in each batch to the minimum length possible – the length of the longest sequence in the batch. To avoid unnecessary calculations on the paddings of sequences, we utilized Pytorch's pack_padded_sequence function.

The architecture design process involved, at first, benchmarking a "simple" LSTM network. The network input included solely the amino acids sequences according to the specified batch size. To process textual representation of the sequences (amino acid types) in a neural network, we have used an embedding layer which transformed the sequences into numerical vectors according to semantics auto detection mechanism. Then, the numerical vector was fed into an LSTM network, comprising 2 layers with a hidden dimension of 128. To avoid overfitting, an additional dropout layer was implemented. After this layer, dimension reduction was realized using a linear dense layer, and finally a sigmoid function was applied to produce the final output. The optimizer used was Adam.

In the next architecture we explored, depicted in Fig.2.a, five additional numeric features were calculated for each amino acid and added to the input of the NN. The corresponding five numerical vectors were concatenated with the numerical vectors of the embedded sequences and fed together into the LSTM's input layer. The features were added to assist with the NN convergence.

A variant of this NN architecture, depicted in Fig.2.b, included adding the numerical features after the dropout layer, i.e., without passing them through the LSTM layer at all [12].

Next, we improved the potential of these NNs by enabling the bidirectional LSTM option. This type of LSTM NNs learns from the input sequence in both its forward and backward directions. The final sequence interpretation is based on both forward and backward learning passes. Effectively, is means training two LSTMs where one receives the normal input and the other, it's reversed order. This can provide additional context to the network, as it makes full use of the network's memory and the neighbors' influence. The output then provides a prediction which relies on the neighbors on both sides of the residue.

Due to unsatisfying study results on long sequences and evidence from online research [13] that the network works better on short sequences, we decided to split the sequences into a predetermined window size (see Fig.2.c.). For that purpose, we have implemented a window slicing mechanism with which the NN processed 256 long sequences at a time (i.e., 256 residues). The mechanism was tested with an overlap between windows, and without. Furthermore, the effect on the model's loss was examined, which was calculated from the model optimizer. For that, two variations were tested: the first, updated the network's weights after each window (i.e., after processing 256 residues); the second, after each batch (i.e., after processing all windows of each sequence in the batch).

As a last step, we added the option to select between the two discussed types of RNN networks. Currently, the model supports both LSTM and GRU based architecture. Support of other RNN architectures can easily be added in the future.

Throughout the process, we tried to optimize the hyper parameters of each selected network. These hyper parameters include the number of batches, the size of the hidden dimension, the number of layers in the network, the learning rate, the number of epochs, and the embedding size.

2.3 Loss Function

Since we desire to predict per residue whether it is in an epitope, we use the binary cross-entropy loss function ("BCELoss" in Pytorch), between the target and the output. The function is used to calculate losses for problems of binary classification. Here, the two classes are epitope and non-epitope, per residue. This is a private case of the multi-class classification, compatible for more than two classes.

Alongside the "BCELoss" function, we examined Pytorch's "CrossEntropyLoss" function, which performs the calculation for a multi-class case. The results and conclusions regarding the two possibilities considered will be discussed in the following chapters.

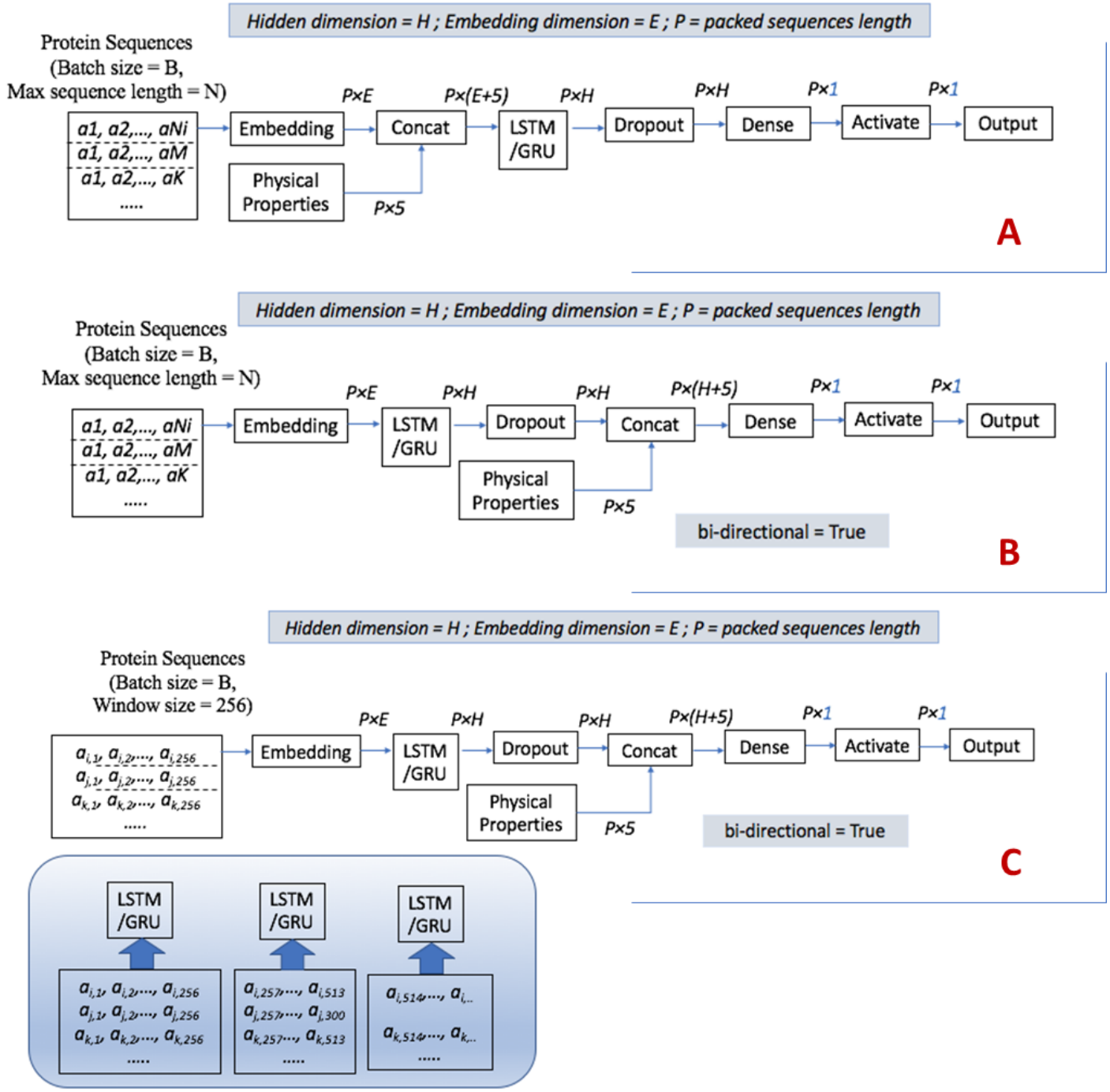


Fig. 2. Scheme of three architectures of the neural network. All architectures provide a choice between LSTM and GRU. All three of them are comprised of an embedding of the amino acid type first, then RNN, dropout, dense layer, activation, and finally the output. Keep in mind that the sequences are of variable length, from a few dozen up to several thousand. The variable length is shown to the left where a sequence of length M appears as $(a_1, a_2, \ldots, a_M)$ and a_i is the ith residue. (a) Sequences are given at their full length; physical properties are concatenated to the amino acid type before the LSTM and inputted altogether; a bi-directional scan of the data is optional. (b) Sequences are given at their full length; physical properties are concatenated after the LSTM; a bi-directional scan of the data is enforced. (c) General structure is as in (b) but sequences are split into windows of size 256 residues, with or without overlap between them. Architecture (c) was tested in two forms: in the first, the network was updated after each window of 256 residues; in the second, the network was updated at the end of each batch only. To the bottom left is a visualization of the windows with no overlap. For example, for a sequence of length M a prediction will be given to $(a_1, a_2, \ldots, a_{256})$ first, then $(a_{257}, a_2, \ldots, a_{513})$ etc. As mentioned earlier the loss calculation and weights update can be done at the end of each window or at the end of each batch.

2.4 Optimization Hyperparameters

Training a neural network is a delicate procedure, which often requires achieving optimization of numerous hyperparameters. Finding optimal values for the hyperparameters is of major importance and can significantly affect the training results. To get a quality estimation of the optimal values, we have tried two different tools. Either tool has a very similar way of work. In general lines, each hyperparameter can be given either a continuous or a discrete range of values, and a preferable manner of picking the sampled values (uniformly for example).

The tool chooses a permutation of the hyperparameters we have selected and runs the program. The process repeats itself several times, determined by a trial number parameter given to the tool. Moreover, the tools have an inner optimization that can "kill" active permutation running if it does not seem to yield successful results. The decision to stop a certain trial is based upon previous results and comparison to them. Both tools are run using a GPU.

The first tool examined is called "Ray-tune" [14]. At first, to get a sense of the hyperparameters' order of magnitude, we have run the program on meager-sized data. This tool's advantage is that it has a very detailed output. An example is shown in Fig. 3.

```
-- Status --
Memory usage on this node: 4.9/12.7 GiB
Using AsyncHyperBand: num_stopped=10
Bracket: Iter 32.000: -0.23898763954639435 | Iter 16.000: -0.25457823276519775 | Iter 8.000: -0.2935655415058136 | Iter 4.000: -0.613279998
Resources requested: 0/2 CPUs, 0/1 GPUs, 0.0/7.32 GiB heap, 0.0/3.66 GiB objects (0.0/1.0 accelerator_type:K80)
Result logdir: /root/ray_results/DEFAULT_2021-08-31_20-48-27
Number of trials: 10/10 (10 TERMINATED)
+--------------------+------------+-------+--------------+-----------+--------------+-------------+--------------+----------+------------+
| Trial name         | status     | loc   |   batch_size |   dropout |   hidden dim |          lr |   num layers |     loss |   accuracy |
|--------------------+------------+-------+--------------+-----------+--------------+-------------+--------------+----------+------------|
| DEFAULT_cab1e_00000 | TERMINATED |       |            8 | 0.167816  |          128 | 0.00014377  |            2 | 0.217588 |  0.670207  |
| DEFAULT_cab1e_00001 | TERMINATED |       |           16 | 0.0701064 |          512 | 0.002204    |            4 | 0.415127 |  0.502204  |
| DEFAULT_cab1e_00002 | TERMINATED |       |            8 | 0.173341  |          128 | 0.0108199   |            4 | 0.571228 |  0.427326  |
| DEFAULT_cab1e_00003 | TERMINATED |       |            2 | 0.200201  |           64 | 0.000745648 |            4 | 0.422076 |  0.485371  |
| DEFAULT_cab1e_00004 | TERMINATED |       |           16 | 0.17968   |          512 | 0.0329351   |            3 | 0.820136 |  0.175663  |
| DEFAULT_cab1e_00005 | TERMINATED |       |            4 | 0.111086  |          256 | 0.0885444   |            4 | 0.96892  |  0.0302503 |
| DEFAULT_cab1e_00006 | TERMINATED |       |            2 | 0.125874  |          512 | 0.0157602   |            3 | 0.69837  |  0.302877  |
| DEFAULT_cab1e_00007 | TERMINATED |       |            8 | 0.174021  |           64 | 0.0103272   |            2 | 0.61328  |  0.375786  |
| DEFAULT_cab1e_00008 | TERMINATED |       |            2 | 0.0872315 |          256 | 0.020621    |            3 | 0.399666 |  0.47963   |
| DEFAULT_cab1e_00009 | TERMINATED |       |            8 | 0.0665964 |          256 | 0.000117355 |            2 | 0.402842 |  0.498235  |
+--------------------+------------+-------+--------------+-----------+--------------+-------------+--------------+----------+------------+

2021-08-31 20:49:48,902 INFO tune.py:561 --
Best trial config: {'lr': 0.00014376974875789883, 'batch_size': 8, 'hidden dim': 128, 'num layers': 2, 'dropout': 0.16781609967569489}
Best trial final validation loss: 0.21758848428726196
Best trial final validation accuracy: 0.6702070236206055
```

Fig. 3. An example of the output received from the "Ray-tune" tool. The table shows the hyperparameters tested and the loss and accuracy rates for each trial. Optimization can be done to minimize losses or maximize accuracy.

As we advanced, we wanted to enlarge the data used for the optimization of the hyperparameters. Unfortunately, the Ray-tune tool was unable to handle the data size, and we had to re-wrap our code with a different tool. At that point, we searched for a new tool and found the "Optuna" tool [15]. As mentioned, the tools' functionality is quite similar. Indeed, the second tool supported large datasets and we were able to continue. The hyperparameters chosen for optimization were the batch size, dropout value, hidden layer dimension, embedding size, number of layers, RNN architecture (LSTM / GRU), and the learning rate. Additionally, we let the tool find the optimal window size if any (i.e., whether we should slice the sequences or not, and to which size).

3 Experimental Results

3.1 Benchmarking

Since no convergence was observed for a large amount of data in any of the discussed neural network architectures, we decided to conduct tests with smaller chunks of data. This was done primarily to verify the model's correctness. We surmised that if training is successful for a few inputs, the architecture is reasonable. In such case, we would conclude that the implementation has no bugs, although it might require further optimization.

Hence, we evaluated the various architectures described herein by training on a dataset of ten sequences (non-shuffled) over several epochs and comparing their performances.

Performance was estimated by the loss decrease rate, the achieved accuracy, and additional metrics. Since we found that even after applying a union between recurrent sequences in the database most of the amino acids are not classified as epitopes, the model might tend towards high accuracies despite under-predicting epitopes. Therefore, we included recall and precision metrics for better representation of the model performance. Recall measures the ratio of the correctly predicted epitopes (true positive, TP) to the total number of true epitopes (true positive plus false negative). Precision measures the ratio of the correctly predicted epitopes (true positive, TP) to the total number of predicted epitopes (true positive plus false positive). Another metric we used was the F1 measure, derived from recall and precision. It represents the balance between these properties and is most useful in cases of uneven class distributions such as in the present study.

$$F1 = 2\times\frac{Precision\cdot Recall}{Precision+Recall}$$

The first training attempts we conducted using a single batch of ten sequences, showed that it took on average, for most of the architectures described herein, less than 60 epochs to achieve satisfactory results (high recall and precision in addition to low loss). The F1 scores were in the vicinity of 0.7 which appeared to be relatively close to the value of 1, corresponding to perfect precision and recall.

With that information, we decided to increase the number of batches to 5 for all the architectures while maintaining the same batch size of 10. Ergo, in this configuration the training dataset was of size 50. This configuration emphasized the superior performance of some models relative to the rest. Results indicate that the GRU neural network converges faster than LSTM (with or without a sliding window). One of the possible explanations is that the former uses fewer parameters and therefore less memory (only 2 gates) than the latter.

We then proceeded by further increasing the number of batches to 10, while keeping the other parameters intact, hence working with a training dataset of 100. The results of these tests clearly showed that the GRU networks have superior performance over the LSTM based models.

Graphs are attached in Fig. 4.

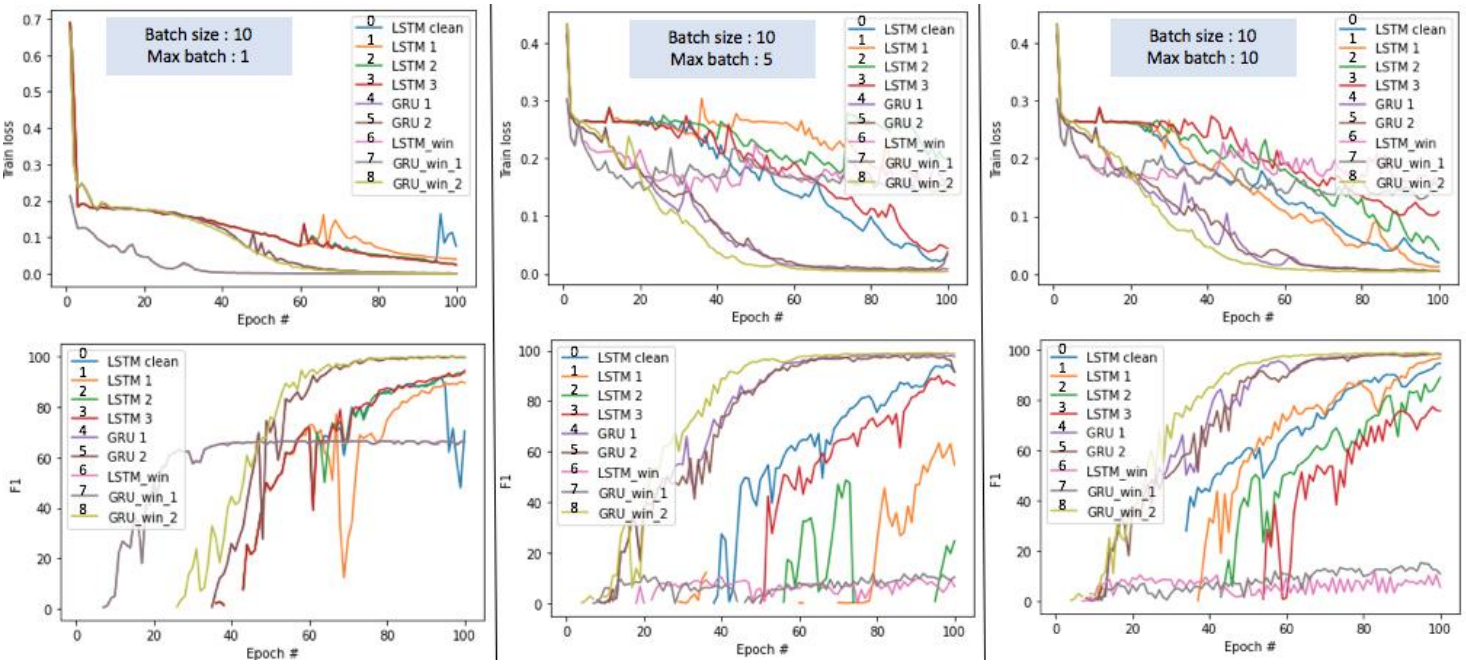


Fig. 4. Loss rates and F1 score for a small dataset, using nine different architectures.

(0) LSTM; (1) LSTM with numerical features as well (physical properties); (2) bi-directional LSTM with numerical features; (3) bi-directional LSTM with numerical features concatenated at the dense layer; (4) bi-directional GRU with numerical features; (5) bi-directional GRU with numerical features concatenated at the dense layer; (6) bi-directional LSTM with numerical features and sliced sequences (windows), with network updates at the end of each window; (7) bi-directional GRU with numerical features and sliced sequences (windows), with network updates at the end of each window; (8) bi-directional GRU with numerical features and sliced sequences (windows), with network updates at the end of the batch (after processing all windows in batch).

3.2 Introducing Weights

In the original dataset and most of our unionized dataset, the majority of residues are tagged as non-epitope. Hence, the data is imbalanced as the class of epitopes is under-represented. And in fact, while trying to train the network we encountered the issue. While observing low loss rates and high accuracy, the output vectors showed the network wrongfully predicts all residues as non-epitopes. Since the positive tags' percentage in the sequence was usually very low, it still received low losses. Eventually, after about 60 epochs, the network was able to predict some epitopes, but this is a considerably long time for such small dataset size (see section 3.1).

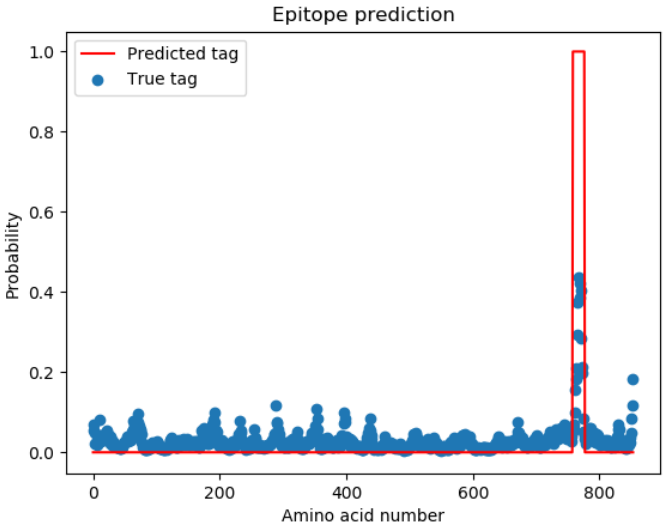


Fig. 5. Training results for a single sequence, after 40 epochs. The red line shows the true tags, and the blue dots are the predicted tags. Indeed, at the area of the positive tags the probability is higher than the rest but is still below 50% and is therefore ruled negative. It is possible that with more epochs these number will rise but training on a single sequence and testing on the same one should produce better results than those.

To compensate for the under-representation of the positive tags, we introduced weights. Giving a higher weight to the positive tags than the negative results in higher losses for a false negative result and enforces the network to adapt. Therefore, we used Pytorch's cross-entropy loss function, compatible multi class classification and, in that, the binary case as well. The function also allows using weights and was therefore suitable for us.

Then, the optimal weights ratio had to be found. To get an educated initial value, we used the reversed ratio of the classes' representation in the sequence. First, we tested the method on a single sequence. The sequence was 855 residues long, out of only 19 in an epitope. The representation ratio is then 1:44 in favor of the negative tags. In accordance, we tested the sequence in the same configuration as before, but with a weights ratio of 1:44 in favor of the positives.

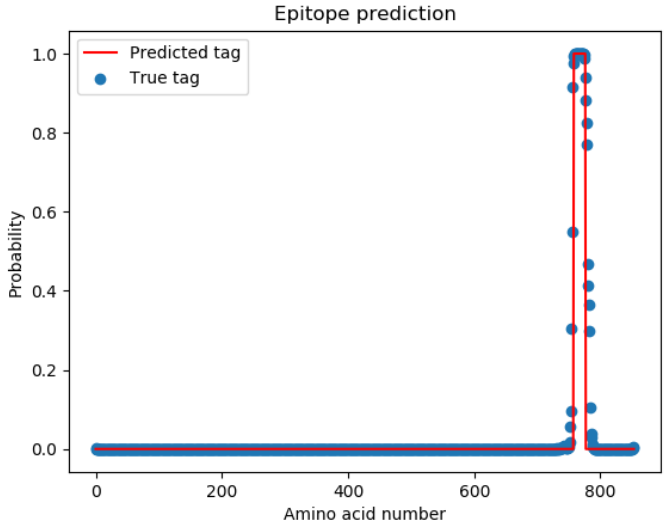


Fig. 6. Training results for the same sequence as in fig. 5, after 40 epochs. Clearly, this is an overfit and the learning process could have been stopped much earlier, but this example is only for the sake of examining the weighting method. The final loss was 32% and the accuracy 99%. As can be seen above, there are a few false negatives which explain the 30% loss (recall that the ratio was 1:44 hence a false negative tag arises severe losses).

Given that the weight ratio fit perfectly to the sequence (as we only used one sequence), the results were very satisfying. However, a single weight ratio had to be applied and fit all sequences. We calculated the epitope's percentage of each sequence in our unionized dataset to get an estimate of the optimal value. A histogram of the results can be seen in fig 7. The median, mean and standard deviation values are:

$$median \cong 7\%$$
$$mean \cong 17\%$$
$$standard\ deviation \cong 25\%$$

Clearly, the data is imbalanced and does not distribute normally. Moreover, the standard deviation is very large given that the range of values is only $(0, 100]$.

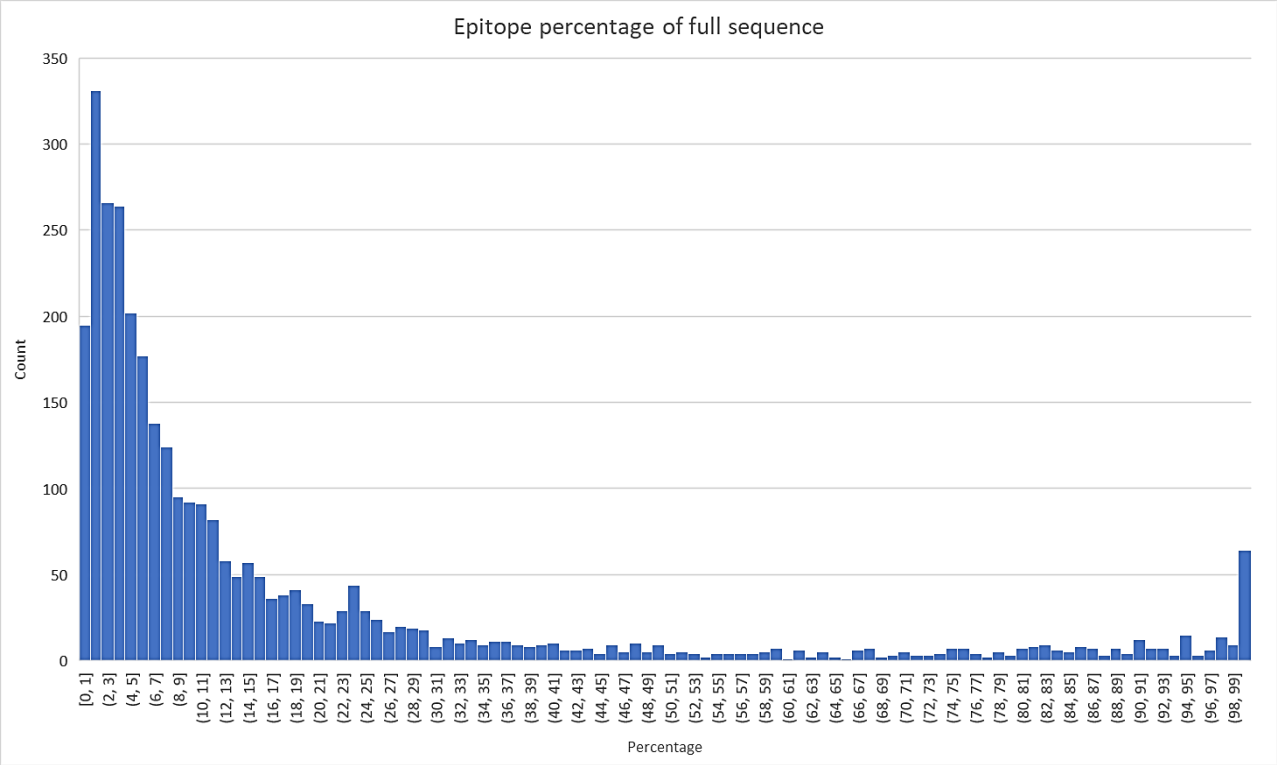


Fig. 7. A histogram of the percentage of epitope residues in a sequence. The horizontal axis shows the percentages from 0 to 100, in bin width of 1 percent. The vertical axis is the count of sequences which showed the percentage. One can easily see that the data is imbalanced and does not seem to distribute normally.

Nevertheless, we tried to see if a weights ratio around the median of the whole data would produce good results. The median value for positive tags percentage was 7%, which translates to a ratio of 1:13. First, we tested the weights' impact on a dataset of 50 samples only, i.e., 35 sequences for training and 15 for testing.

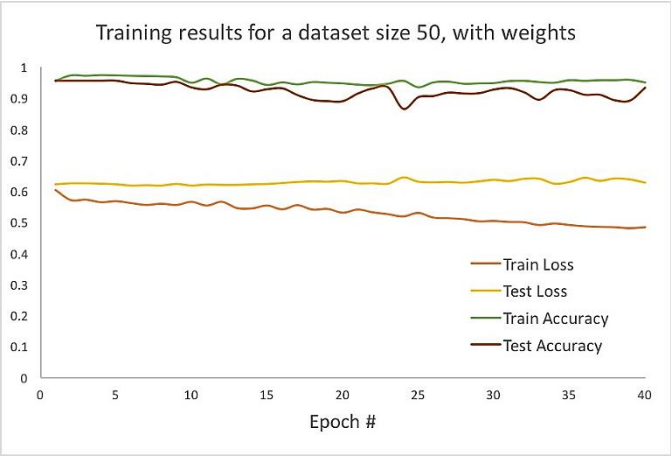


Fig. 8. Training results for a dataset of 35 train sequences and 15 test sequences, along 40epochs. The final training loss was 48% and the accuracy was 95%. The final test rates were 63% loss and 93% accuracy, but these numbers remained almost constant throughout the training.

The behavior of the complete dataset could, of course, be different than this small example. However, since running times for 3000 samples are significantly longer, that was a necessary step to view the weight ratio's benefit. The network was trained along 40 epochs, as in the single sequence examples shown in figures 5 and 6.

As can be seen in Fig. 8, the results are not ideal but present a slight learning process. The training loss reduces through time, but very slowly. The test loss however remains constant. Since the sampled data was very small, it is expected that the test loss will not behave as desired. Both train and test accuracies are too high, from the very beginning of the training process, which suggests the learning is not ideal. A possible explanation for that is that the accuracy does not consider the weights for each tag. Since most expected tags are indeed negative, the accuracy is high.

Nonetheless, the recall and precision values imply that the network does give a more significant value to the positive tags (see Fig. 9). The final recall value was 40% and precision was 27%, meaning there are now positively tagged residues, about 27%

of them are true positives, and about 40% of all expected positive tags are indeed tagged as such. Given that the initial values were about 0% for both rates, we can infer that the network does somewhat learn properly, however very slowly. Note that a weights ratio of 1:13 is the median value for the entire dataset and not for the small example tested. Therefore, it is understandable that results may not be ideal.

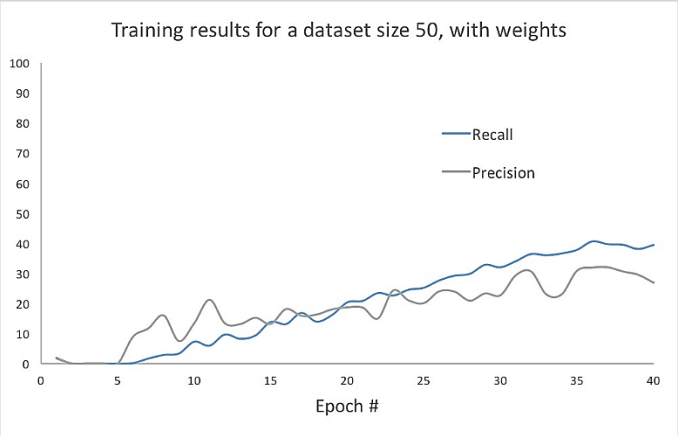


Fig. 9. Recall and precision rates for the training dataset, throughout the 40 epochs. The curves present a learning process as the general trend is upwards. While not ideal and were to be much lower after 40 epochs given the small data, the final recall and precision rates suggest a positive effect of the weights. One can infer that applying weights indeed pushes the network to estimate more tags positively.

Due to these results, we decided to train the network on the complete dataset, with that same weights ratio of 1:13. Since we were not very confident that this configuration will be successful, we decided to train the network with a second configuration as well and compare the results. The second was identical, however, used the BCELoss, i.e., with no weights.

3.3 Hyperparameters' Values

As explained in section 2.4, the hyperparameters were chosen via an external tool. Eventually, the following values were chosen:

$$Embedding\ size = 30, \quad Batch\ size = 15$$
$$Hidden\ dimension = 256, \quad Dropout = 15$$
$$Number\ of\ layers = 2, \quad RNN = GRU$$
$$Learning\ rate = 3.3 \times 10^{-5}, \quad No\ windows$$

3.4 Final Training Outcome

Ultimately, the final network was trained on the complete dataset (3158 sequences) using the "Google Colaboratory" platform, which allows running Python code online, using Google's GPU services [16]. Two training configurations were tested, which differ in the loss function only. One was run using the BCELoss function, i.e., with no weights for the positive vs negative classes. The second was identical but used the CrossEntropyLoss with a weight ratio of 1:13 as described in the previous section. Both were run for 50 epochs, and the models' state was saved after each epoch in case the session will be terminated mid-way. Indeed, the training without weights was shut down after epoch 17. We attempted to load the final state and resume training from that point, but the user permissions to use the GPU were terminated due to Google's policy. On the other hand, the second configuration, with the weights, was successfully executed through all 50 epochs.

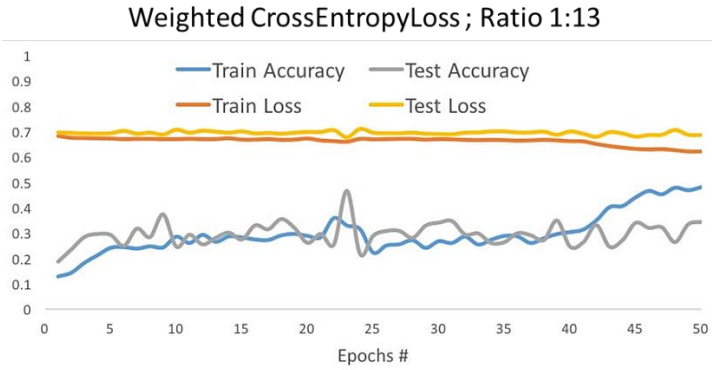


Fig. 10. Training results on the entire dataset throughout 50 epochs, with weights. Note that the changes along the epochs are very mild and the learning process does not seem fast enough.

From Fig. 10, one can note that the learning rate might be too small. Towards the final epoch, there seems to be an improvement in the training dataset results, however not yet for the test dataset. This could imply either that more epochs are needed, or perhaps the beginning of an overfit trend. Unfortunately, based on these 50 epochs it is too soon to tell. Since all states along the run were saved, we decided to keep the results of epoch number 23 as our final training results, as they present the highest test accuracy and the lowest test loss.

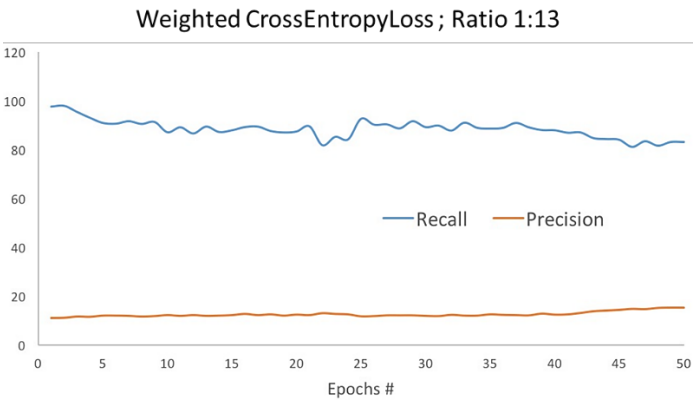


Fig. 11. Recall and precision rates for the training dataset. Results were based on the entire dataset throughout 50 epochs, with weights. Again, the changes along the epochs are very mild. However, these rates imply that the network does not zero all output entries and indeed classifies some regions as epitope and some as non-epitope.

The rates achieved after epoch #50, with weights:

$$Train\ loss = 0.62, \quad Train\ accuracy = 0.48$$
$$Train\ recall = 83\%, \quad Train\ precision = 16\%$$
$$Test\ loss = 0.69, \quad Test\ accuracy = 0.35$$

The rates achieved after epoch #23, with weights:

$$Train\ loss = 0.66, \quad Train\ accuracy = 0.33$$
$$Train\ recall = 85\%, \quad Train\ precision = 13\%$$
$$Test\ loss = 0.68, \quad Test\ accuracy = 0.47$$

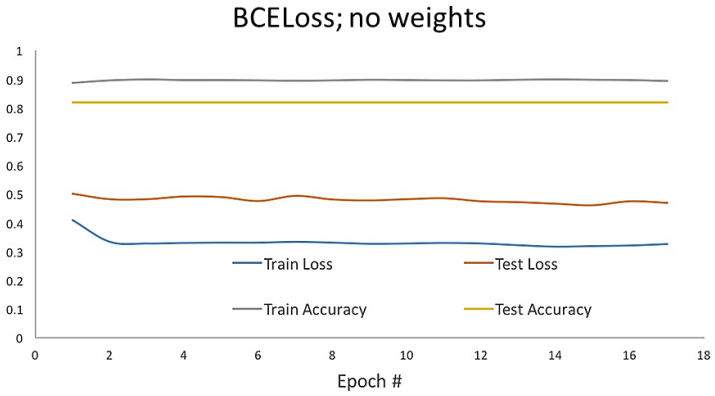


Fig. 12. Training results on the entire dataset throughout 50 epochs, without weights. Note that changes along the learning process hardly exist.

The results for the training without weights, using the BCELoss, were not very promising either. Fig. 12 shows hardly any changes between epochs, although this might have changed had we had more than 17 epochs. The accuracy rates for both training and test datasets are very high, however, this is again due to the network's tendency to zero all output entries, as is clearly shown in Fig. 13.

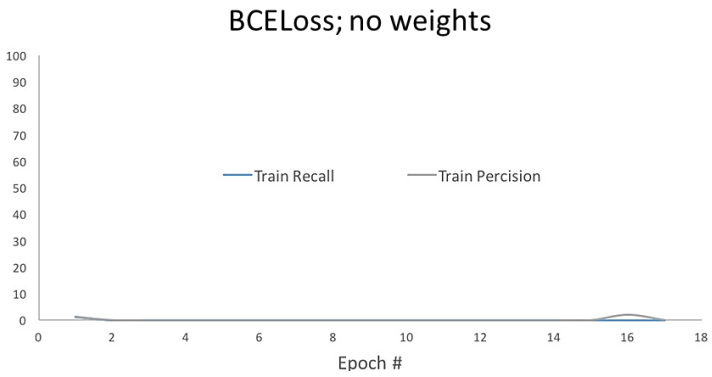


Fig. 13. Recall and precision rates for the training dataset. Results were based on the entire dataset throughout 50 epochs, without weights. The results remained 0% along the entire run, meaning no epitopes were found whatsoever.

It is possible that given more epochs, the network using the BCELoss would have improved and find epitopes. Note that training on a small dataset required about 60 epochs to achieve good training results, therefore 17 epochs for the entire dataset with the same configuration might be too little.

Since the learning rate seemed to be too small, we attempted to train the network with the weighted loss function and a higher learning rate of 5×10^{-4}. Unfortunately, that session was shut down by Google as well, after only 8 epochs. Comparing the first 8 epochs of the two weighted loss training (each with a different learning rate), the results did not seem to differ. Possibly, a longer training would provide different results, but as mentioned before we were unable to use the GPU anymore.

After examining all training results, we concluded that based on all training results we had, the weighted configuration was best, and saved results of epoch #23 for future prediction purposes.

3.5 Visualized Examples

A few examples are shown below, to give a good visualization of the network's prediction capabilities. The examples were taken from PDB files, which can be plotted in 3D using "Chimera" [17]. The proteins in the PDB files were converted to a FASTA file, as the algorithm requires [18]. These sequences were given as input to the trained network and a prediction

was received. Based on that prediction the relevant region in the 3D plot was colored red and is shown below. Each protein is presented with its id. The PDB files can be found in [19].



Fig. 14. ID 1a7l.



Fig. 15. ID 1qr1.



Fig. 16. ID 3i6l.

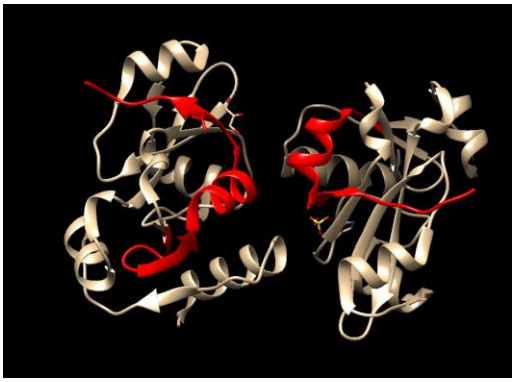




Fig. 17. ID 3ri0. (Two different views of the 3D plot).



Fig. 18. ID 4nt6.

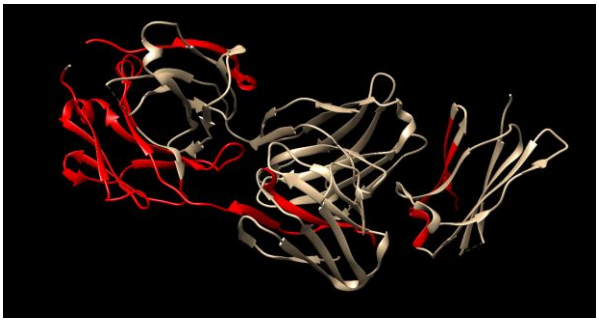


Fig. 19. ID 6ddm.

3.6 Runtime Evaluation

Following is a summary of the running times of several configurations, with several different batch sizes. All times presented refer to the use of Google Colab's free GPU.

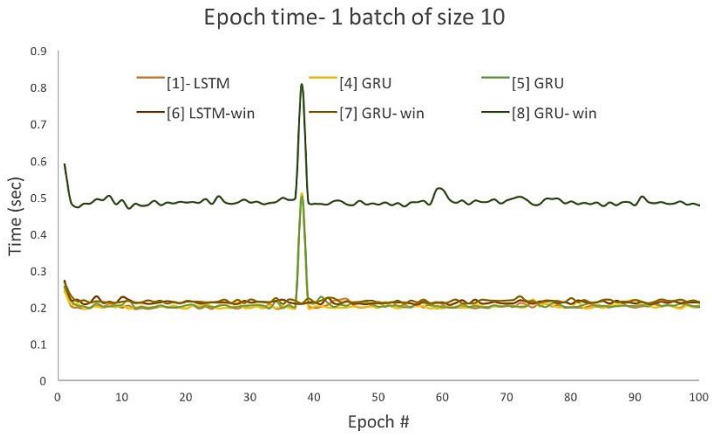


Fig. 20. Epoch run time for a single batch of 10 sequences. All architectures had roughly the same run time per epoch, about 0.2 seconds, when the slowest was the GRU with the sliding window.

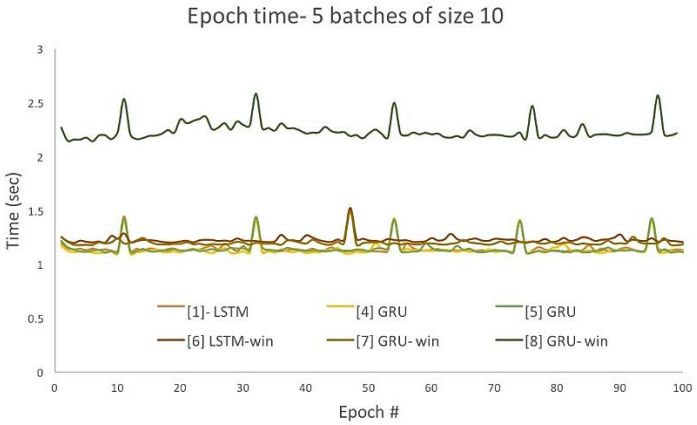


Fig. 21. Epoch run time for 5 batches of 10 sequences. All architectures had roughly the same run time per epoch, about 1.1 seconds, when the slowest was again the GRU with the sliding window.

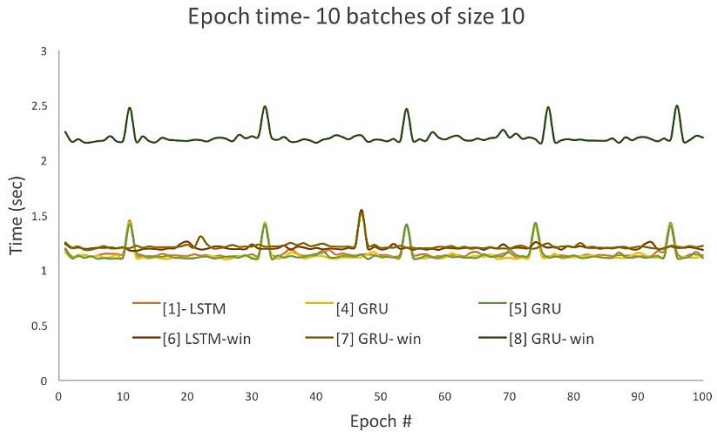


Fig. 22. Epoch run time for 10 batches of 10 sequences. All architectures had roughly the same run time per epoch. Note that these results are very similar to the results for the 5 batches per epoch.

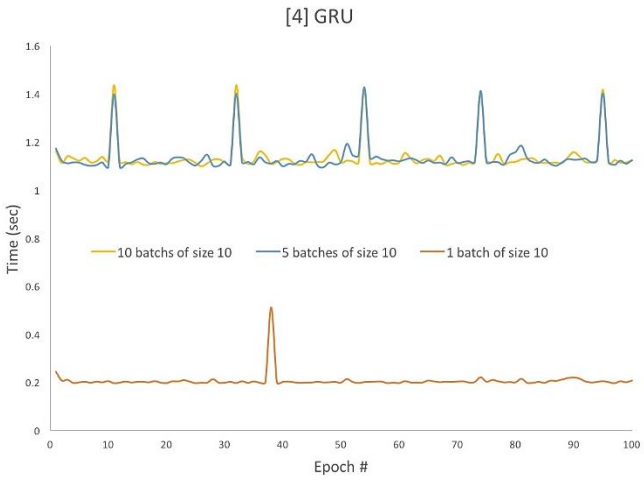


Fig. 23. Epoch run time for different numbers of batches, all of 10 sequences per batch. The run times presented are based on the best architecture we examined, the GRU without any sliding window. This is the architecture which was ultimately chosen for the training of the whole dataset.

The figures above show similar training times for almost all architectures. Some samples of the chosen architecture's runtime are presented in Fig. 23. Table 1 shows the average epoch time for the three configurations we attempted to train, described in section 3.4. One can note that all times in the table are quite similar.

Average epoch time ; train mode; all DB	
CrossEntropyLoss_WeightsRaio_1to13_lr33e-6	219.538 sec
CrossEntropyLoss_WeightsRaio_1to13_lr5e-4	227.400 sec
BCELoss_lr_33e-6	217.950 sec
BCELoss reference	220.100 sec

Table 1. A comparison between the average epoch training times on the complete database for a few selected architectures. The first entry corresponds with the network architecture using cross entropy loss function with different weights assigned to each of the classes and with a learning rate of 33e-6. The second entry is the same as the first one except for a higher learning rate of 5e-4. The third entry corresponds with the BCELoss function and a learning rate of 33e-6 as described above. The last entry corresponds with the basic non-optimized architecture using BCELoss, learning rate of 1e-2, a hidden dimension of 128, 2 layers, and embedding size of 27.

Average epoch time ; train mode;	
10 sequences	0.21 sec
50 sequences	1.16 sec
100 sequences	1.16 sec
All DB	220.1 sec

Table 2. A comparison between the average epoch training times on a smaller subset of the dataset using the same basic BCELoss based architecture from Table 1.

4 Conclusions

In this paper, we covered several possible configurations of an RNN for the linear epitope prediction in proteins problem. First, the various options were tested on just a few sequences, to verify the network's ability to learn them. We discovered that the general architecture we built is indeed capable of learning, although many epochs were necessary for that. We tried using an LSTM and a GRU, enabling a bi-directional scan, slice the sequences or give them as a whole and concatenate the physical properties before the RNN and after. All possibilities proved fit for a small dataset, with not much of a difference in their outcome and runtime. Ultimately, we chose the architecture that was slightly better than the rest, which was the GRU, without windows, a bi-directional scan, and a concatenation of the numerical features after the RNN. Moreover, we used standardization of the numerical features to try and shorten the time necessary for convergence. After choosing the exact architecture, we noticed the number of epochs it takes the network to find epitopes and not rule all residues as non-epitope. We introduced a weighted loss function to differentiate the positive and negative tags and explored the correct ratio necessary for that. When training on the entire dataset, the choice of a weighted loss function seemed to be reasonable, however, accurate conclusions were hard to be made due to technical issues which prevented us from training the non-weighted network sufficiently. Finally, we observed the learning curve of the training process and concluded that a higher learning rate might be beneficial.

Several improvements are worth testing in future work, based on our findings. First, after merging identical sequences in the dataset, we learnt that there remain sequences which are not identical, but highly similar. Clustering these sequences might provide some better results, as was done in previous work [4]. Alternatively, one might consider training the network without merging the identical sequences prior to that and compare those results. It is possible that our trained network predicts some of the epitope possibilities but not all, and since each sequence is a union of several observations of epitopes, the accuracy is too low (i.e., the network finds some epitopes in the sequence but not all). Due to lack of time and the long training runtimes, we did not get to make that comparison. Moreover, if a weighted loss is not sufficient, or if a single weight ratio to fit all sequences is hard to find, implementing under sampling may improve results. As for the RNNs ability to learn long sequences, some suggest that inputting the sequences in order of length, from short to long, might help with the learning process [13c]. Another approach is to test the initialization of the network's weights and bias. Accurate and cautious initialization may improve the results [20]. Additionally, since the secondary structure was inputted as a numerical feature, there is some room for improvement there. Since the network might learn that a secondary structure of 0 is close to a

secondary structure of 1, an embedding layer for the secondary structure might prove beneficial. Unfortunately, this is another improvement we did not get to examine due to lack of time.

Lastly, if all of the above proves insufficient, replacing RNNs with another type of NN can achieve the desired results, for example CNNs [13b].

5 References

[1] Britannica, T. Editors of Encyclopaedia (2019, February 18). Epitope. Encyclopedia Britannica. https://www.britannica.com/science/epitope.

[2] Sepideh Parvizpour, Mohammad M. Pourseif, Jafar Razmara, Mohammad A. Rafi, Yadollah Omidi, Epitope-based vaccine design: a comprehensive overview of bioinformatics approaches, Drug Discovery Today. (Volume 25, Issue 6, 2020, Pages 1034-1042). https://www.sciencedirect.com/science/article/pii/S1359644620301136

[3] BepiPred-2.0: Sequential B-Cell Epitope Predictor. http://www.cbs.dtu.dk/services/BepiPred/download.php

[4] Maximilian Collatz, Florian Mock, Emanuel Barth, Martin Hölzer, Konrad Sachse, Manja Marz, EpiDope: a deep neural network for linear B-cell epitope prediction, Bioinformatics, Volume 37, Issue 4, 15 February 2021, Pages 448–455, https://doi.org/10.1093/bioinformatics/btaa773

[5] Charles C. Bigelow, On the average hydrophobicity of proteins and the relation between it and protein structure, Journal of Theoretical Biology, Volume 16, Issue 2, 1967, Pages 187-211, ISSN 0022-5193, https://doi.org/10.1016/0022-5193(67)90004-5. https://www.sciencedirect.com/science/article/pii/0022519367900045

[6] Sweet RM, Eisenberg D. Correlation of sequence hydrophobicities measures similarity in three-dimensional protein structure. J Mol Biol. 1983 Dec 25;171(4):479-88. doi: 10.1016/0022-2836(83)90041-4. PMID: 6663622.

[7] Radzicka, Anna & Wolfenden, Richard. (1988). Comparing the polarities of the amino acids: Side-chain distribution coefficients between the vapor phase, cyclohexane, 1-octanol, and neutral aqueous solution. Biochemistry. 27. 10.1021/bi00405a042.

[8] Tien, M. Z., Meyer, A. G., Sydykova, D. K., Spielman, S. J., & Wilke, C. O. (2013). Maximum allowed solvent accessibilites of residues in proteins. *PloS one*, *8*(11), e80635. https://doi.org/10.1371/journal.pone.0080635

[9] Biopython module. https://biopython.org/docs/1.75/api/Bio.SeqUtils.ProtParam.html

[10] Standardization of data. https://machinelearningmastery.com/how-to-improve-neural-network-stability-and-modeling-performance-with-data-scaling/

[11] Pytorch module. https://pytorch.org/

[12] Combining numerical features to textual data. https://towardsdatascience.com/combining-numerical-and-text-features-in-deep-neural-networks-e91f0237eea4

[13] RNNs lesser results for long sequences:
(a) https://machinelearningmastery.com/handle-long-sequences-long-short-term-memory-recurrent-neural-networks/
(b) https://towardsdatascience.com/the-fall-of-rnn-lstm-2d1594c74ce0
(c) https://medium.com/mindboard/training-recurrent-neural-networks-on-long-sequences-b7a3f2079d49

[14] Ray tune documentation https://docs.ray.io/en/latest/tune/index.html

[15] Optuna documentation and examples.
(a) https://optuna.org/
(b) https://github.com/optuna/optuna-examples/blob/main/pytorch/pytorch_simple.py

[16] Google “Colab”. https://colab.research.google.com/notebooks/intro.ipynb?utm_source=scs-index#scrollTo=5fCEDCU_qrC0

[17] Chimera documentation. https://www.cgl.ucsf.edu/chimera/

[18] Converting PDB to FASTA. https://zhanggroup.org/pdb2fasta/

[19] PDB website. https://www.rcsb.org/

[20] NNs weight initialization. https://towardsdatascience.com/what-is-weight-initialization-in-neural-nets-and-why-it-matters-ec45398f99fa